

Function call vs System call

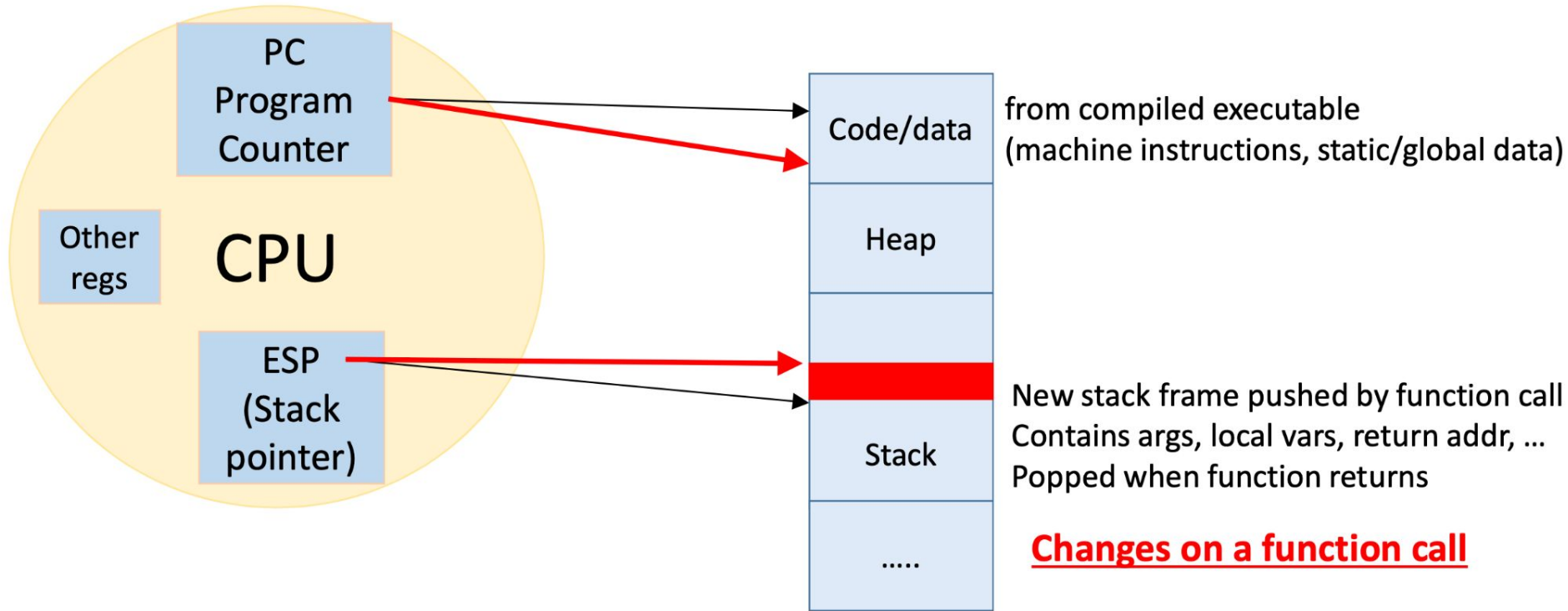
What happens when a user program makes a function call?

- Allocate memory on user stack for function arguments, local variables, ..
- Push return address, PC jumps to function code
- Push register context (to resume execution when function returns)
- Execute function code
- When returning from function, pop return address, pop register context

System call also must

- Use a stack to push register context
- Save old PC, change PC to point to OS code to handle system call
- Run system call, restore context back to user code

Function Call



Located at some memory addresses in RAM

System Call

What is different for a system call?

Changing PC in function call vs. system call

- In function call, address of function code known in executable, can jump to function code directly using a CPU instruction (“call” in x86)
- For system call, cannot trust user to jump to correct OS code (what if user jumps to inappropriate privileged code?)

Saving register context on stack in function call vs. system call

- In function call, register context is saved and restored from user stack
- For system call, OS does not wish to use user stack (what if user has setup malicious values on the stack?)

We require: a **secure stack**, a secure way of jumping to OS code

Kernel Stack and IDT

Every process uses a separate **kernel stack** for running kernel code

- Part of PCB of process, in OS memory, not accessible in user mode
- Used like user stack, but for kernel mode execution
- Context pushed on kernel stack during system call, popped when done

To set PC, CPU accesses **Interrupt Descriptor Table (IDT)**

- Data structure with addresses of kernel code to jump to for events
- Set up by OS during boot up, not accessible in user mode
- CPU uses IDT to locate address of OS code to jump to

Together: secure way of locating OS code, secure stack for OS to run

Trap instruction

When user code wants to make system call, it invokes special “trap instruction” with an argument

- Example: “int n” in x86, argument “n” indicates type of trap (syscall, interrupt)
- The value of “n” specifies index into IDT array, which OS function to jump to

When CPU runs the trap instruction:

- CPU moves to higher privilege level
- CPU shifts stack pointer register to kernel stack of process
- Register context is saved on kernel stack (part of PCB)
- Address of OS code to jump to is obtained from IDT, PC points to OS code
- OS code starts to run, on a secure stack

Why do we need trap instruction?

Need a secure way of jumping to OS code to handle traps

- User code cannot be trusted to jump to correct OS code
- Only CPU can be trusted to handover control from user to OS securely

Who calls trap instruction?

- System call code in a language library (printf invokes system call via int n)
- External hardware raises interrupt, causes CPU to execute "int n"
- Argument "n" indicates whether system call /IRQ number of hardware device

Across all cases, the mechanism is: save context on kernel stack, switch to OS address in IDT, run OS code to handle trap

Return from trap

When OS is done handling syscall or interrupt, it calls a special instruction return-from-trap

- Restore context of CPU registers from kernel stack
- Change CPU privilege from kernel mode to user mode
- Restore PC and jump to user code after trap

User process unaware that it was suspended, resumes execution at the point it stopped before

Always return to the same user process from kernel mode? No

- Before returning to user mode, OS checks if it must switch to another process

Function call vs System call

- Function call: application calls another function.
- System call: application requests a service from the OS.

	Normal function call	System call
Called by	Program	Program
Executes in	User code	Kernel code
Privilege	User mode	Kernel mode
CPU privilege transition	No	Yes
Kernel involved	No	Yes
Example	<code>add()</code>	<code>fork()</code> , <code>read()</code> , <code>write()</code>
Why?	Reuse application code	Request OS service

CPU Scheduling Policies

Why switch between processes?

Sometimes when OS is in kernel mode, it cannot return back to the same process that was running in user mode before

- Process has exited or must be terminated (e.g., segfault)
- Process has made a blocking system call

Sometimes, the OS does not want to return back to the same process

- The process has run for too long
- Must timeshare CPU with other processes

In such cases, OS performs a context switch from one process to another

- Switch from kernel mode of one process to kernel mode of another
- OS scheduler decides which process to run next and switches to it

OS scheduler

- OS maintains list of all active processes (PCBs) in a data structure
 - *Processes added during fork, removed after clean up in wait*
- OS scheduler is special code in the OS that periodically loops over this list and picks processes to run
- Basic outline of scheduler code
 - *When invoked, save context of currently running process in its PCB*
 - *Loop over all ready/runnable processes and identify a process to run next*
 - *Restore context of new process from PCB and get it to run on CPU*
 - *Repeat this process as long as system is running*

OS Scheduler

OS scheduler schedules process on CPU

- One process at a time per core
- Multiple processes can run in parallel on multiple cores
- Scheduling policy: which one of the ready/runnable processes should be run next on a given CPU core?
- Mechanism of context switching (save context of old process in its kernel stack/PCB and restore context of new process) is independent of policy
- Simple scheduling policies have good theoretical guarantees, but not practical for real operating systems
- Real-life schedulers are very complex, involve many heuristics

Scheduling and context switching

- OS scheduling involves two tasks
 - *Policy to pick which process to run*
 - *Mechanism to switch to that process*
- Non preemptive (cooperative) schedulers are polite
 - *Switch only if process blocked or terminated*
- Preemptive (non-cooperative) schedulers can switch even when process is ready to continue
 - *CPU generates periodic timer interrupt*
 - *After servicing interrupt, OS checks if the current process has run for too long*

Preemptive vs. non preemptive schedulers

When is the OS scheduler invoked to trigger a context switch?

Only when a process is in kernel mode for a trap, but not on every trap

Non-preemptive scheduler performs only voluntary context switches

- • Process makes blocking system call
- • Process has exited or has been terminated
- Preemptive scheduler performs involuntary context switches also
 - • Process can be context switched out even if process is still runnable/ready
 - • OS can ensure that no process runs for too long on CPU, starving others
 - • Modern systems use preemptive schedulers
 - • Process can be context switched out any time in its execution

Timer interrupts

Scheduler (OS) can run only when process is in kernel mode

- What if process never goes to kernel mode? How to perform pre-emptive scheduling?
- Timer interrupts: special interrupts that go off periodically
- Used by OS to get back in control periodically
- Increment clock ticks, do other book keeping
- Run scheduling algorithm if required to do involuntary context switch of currently running process
- Hardware support in the form of timer interrupts essential to implementing pre-emptive scheduling policies

Goals of CPU scheduling policy

Maximize utilization: efficient use of CPU hardware

- Minimize completion time / turnaround time of a process = time from process creation to completion
- Minimize response time of a process = time from process creation to first time it is run (important for interactive processes)
- Fairness: all processes get a fair share of CPU (account for priorities also)
- Low overhead of scheduling policy
- Scheduler does not take too long to make a decision (even with large #processes)
- Scheduler does not cause too many context switches (~1 microsecond to switch)

Simplest policy: First In First Out

Newly created processes are put in a FIFO queue, scheduler runs them one after another from queue

- Non-preemptive: process allowed to run till it terminates or blocks
- When process unblocks, the next run is separate “job”, added to queue again
- That is, if a process comes back after I/O wait, it counts as a fresh CPU burst (CPU burst = the CPU time used by a process in a continuous stretch)
- Example schedule: P1 (1-5), P2 (6-8), P3 (9 to 10)

Process	CPU time needed (units)	Arrives at end of time unit	Execution time slots
P1	5	0	1-5
P2	3	1	6-8
P3	2	3	9-10

Problem with FIFO

- Example: three processes arrive at $t=0$ in the order A,B,C
- Problem: convoy effect (small processes get stuck behind long processes)
- Average turnaround times tend to be high

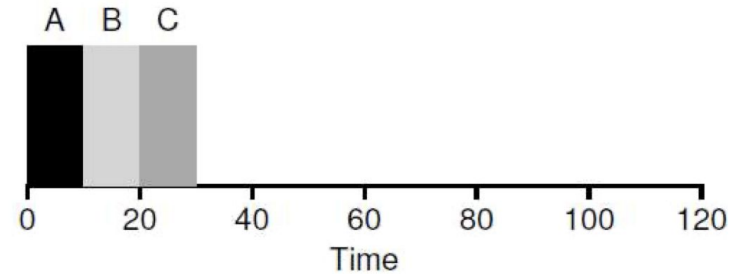


Figure 7.1: FIFO Simple Example

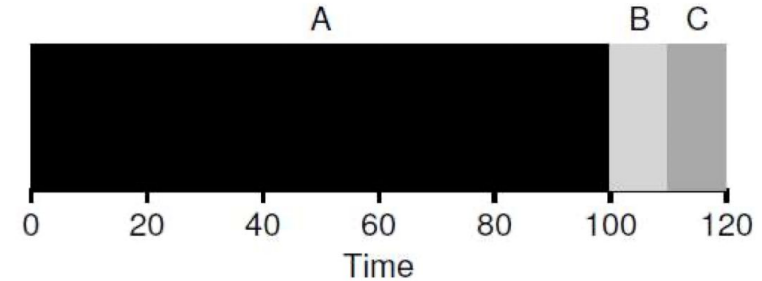


Figure 7.2: Why FIFO Is Not That Great

Shortest Job First (SJF)

Assume CPU burst of a process (amount of time a process runs on CPU until termination/blocking) is known apriori

- Pick process with smallest CPU burst to run next, non-preemptive
- Store PCBs in a heap-like data structure, extract process with min CPU burst
- Example schedule: P1 (1-5), P3 (6-7), P2 (8-10)

Process	CPU burst	Arrival time	Execution time slots
P1	5	0	1-5
P2	3	1	8-10
P3	2	3	6-7

Problem with SJF

- Provably optimal when all processes arrive together
- Theoretically guaranteed to have the lowest average turnaround time across all policies (under certain assumptions)
- SJF is non-preemptive, so short jobs can still get stuck behind long ones.

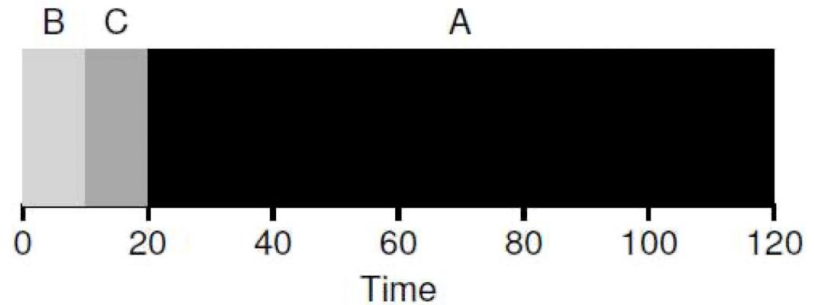


Figure 7.3: SJF Simple Example

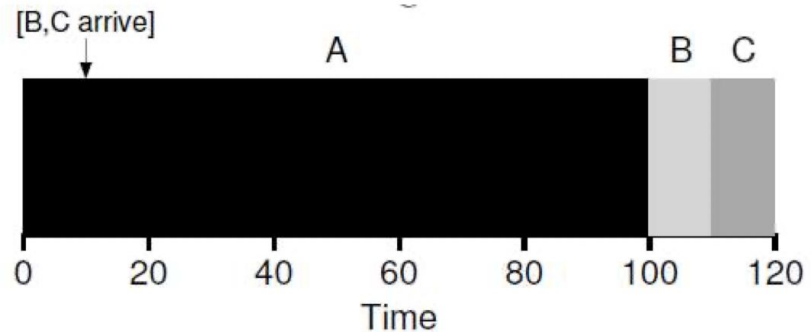


Figure 7.4: SJF With Late Arrivals From B and C

Shortest Remaining Time First (SRTF)

- Preemptive version of SJF A newly arrived process can preempt a running process, if CPU burst of new process is shorter than remaining time of running process
- Avoids problem of short process getting stuck behind long one
- Example schedule: P1 runs for 1 unit, P2 (2-4), P3 (5-6), P1 (7-10)

Process	CPU burst	Arrival time	Execution time slots
P1	5	0	1, preempted , then 7-10
P2	3	1	2-4
P3	2	3	5-6

Round Robin (RR)

- Every process executes for a fixed quantum slice
 - *Slice not too small (to amortize cost of context switch)*
 - *Slice not too big (to provide good responsiveness)*
- Preemptive policy
 - *Timer interrupt used to enforce periodic scheduling*
- Good for response time, fairness
- Bad for turnaround time
- xv6 is a simple RR scheduler

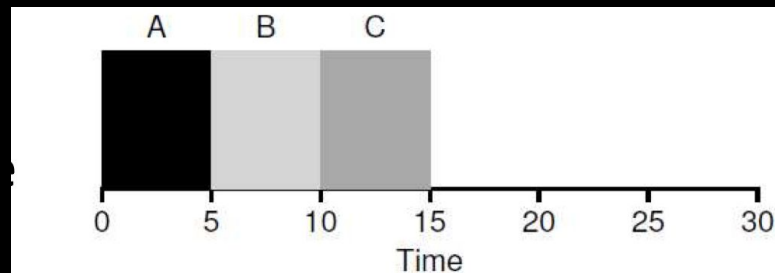


Figure 7.6: SJF Again (Bad for Response Time)

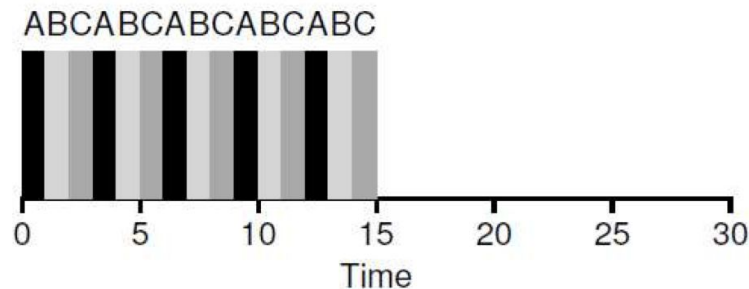


Figure 7.7: Round Robin (Good for Response Time)

Practice Question 1

Q1: Consider the following sequence:

A user starts Application A. The executable is stored on disk. The OS creates a process for A. A is scheduled on the CPU and starts running. While running, A requests data from a file. The OS services the request, and another ready process B gets the CPU.

Explain the OS concepts involved in this sequence.

Practice Question 1

- Program → Process
- Program loading / virtual address space
- System call / user mode → kernel mode
- Process state change and scheduling

Expected Answer

The executable initially exists as a program on disk. When the user runs it, the OS creates a process, including a PID, process state, memory image and execution context. The OS creates its virtual address space and maps the program's code and data into it.

The process starts in a state where it can be scheduled and eventually becomes Running. When Application A requests file data, it uses a system call, causing execution to enter kernel mode so the OS can perform the requested service.

If A must wait for the file I/O, A moves from Running → Blocked/Waiting. The scheduler can then select B, causing B to move from Ready → Running. When A's I/O completes, it can move back to Ready.

These concepts are presented across the slides on program loading, system calls, processes and process states.

Practice Question 2

Q2: A company wants to run three applications on one physical server.

Approach A: Each application runs with its own Guest OS on top of a hypervisor.

Approach B: Each application runs in a separate container while sharing the host OS kernel.

Explain the main architectural difference between A and B. Which approach provides full machine/OS virtualization, and which approach primarily provides process/environment isolation?

Practice Question 2

Expected Answer

Approach A represents virtual machines. Each VM has a Guest OS and runs above a hypervisor. The machine and its OS are virtualized.

Approach B represents containers. Containers run isolated application environments while sharing the host OS kernel. The slides describe containers as isolated environments using namespaces, while VMs virtualize the machine/OS layer.

Therefore:

A → Virtualization / VM

B → Containerization / isolation

Practice Question 3

Q3: Suppose Process P1 is running and the OS decides to stop it temporarily so that another process can run.

Explain what information the OS needs to preserve so that P1 can continue correctly later. Also explain why the entire process does not need to be copied during a context switch.

Practice Question 3

Expected Answer

The OS needs to preserve P1's CPU execution context, including the Program Counter, general-purpose registers, Stack Pointer, and processor status/flags. This information allows the CPU to resume P1 from the correct instruction with the correct register and stack state.

The OS does not need to copy the entire process because the process's code, data, heap and stack generally remain in memory. The main operation is saving P1's CPU state and restoring the state of the next process.

Practice Question 4

Q4: A process is executing in user mode and performs a system call. Later, the scheduler switches the CPU from Process P1 to Process P2.

Explain the difference between these two switches:

User mode → Kernel mode

P1 → P2

Why is the second operation generally more expensive?

Practice Question 4

Expected Answer

A mode switch changes the CPU's privilege level from user mode to kernel mode so that the kernel can execute a privileged service. The same process is still executing; only its execution mode changes.

A context switch changes the process/thread executing on the CPU. The OS must save the execution context of P1 and restore the saved context of P2. This can involve the program counter, stack pointer, general-purpose registers and, for a process switch, address-space-related state.

Therefore, a context switch generally involves more state changes than a simple mode switch.